

GAME PROGRAMMING PATTERNS

マリオに近づける

ダッシュ・ヒップドロップの強化・壁アクションを追加する

対象：3年生

MARIO+

LESSON

マリオに近づける

目標：ステートパターンを使い、動きのバリエーションを増やす



前回の振り返り：今あるState

- IdleState（待機）／WalkState（移動）／JumpState（ジャンプ）
- FallState（降下中）／HipDropState（ヒップドロップ）
- この5つのStateとStateMachineで、キャラクターの基本的な動きはひととおり組み上がっている

POINT

今日はこの5つに、新しい動きを追加してさらに本家マリオへ近づけていきます。

今回のゴール

- ① ダッシュ：一定条件で移動速度が上がる動きを追加する
- ② ヒップドロップの強化：開始・終了それぞれにディレイ（タメ・硬直）を追加する
- ③ 壁キック&壁張り付き：壁にくっついてゆっくり降下し、蹴って反対側へ跳べるようにする

STEP 1

ダッシュを追加する

移動に「速い状態」を1つ増やす



ダッシュとは

- 一定の条件（例：ダッシュボタンを押している間）で、通常より速く移動できる状態
- WalkStateの速度を条件分岐で変えるのではなく、DashStateという新しい状態として切り出す
- 「歩く」と「ダッシュ」は似ているけれど別の状態、と考えるのがステートパターンらしいやり方

DashStateの設計

- Enter : 専用アニメーションが無いので、走りアニメーションをAnimator.speedで1.5倍速にして代用する
- Update : 入力方向へ、一定の加速度でdashSpeedまで少しずつ速度を上げていく
- Exit : Animator.speedを1倍に戻し忘れないように注意する

注意

Exitで速度を戻し忘れると、Dash以外の状態でもアニメーションが速いままになってしまいます。

DashStateの実装①：Enter・Exit

- 走りアニメーションはそのまま使い、再生速度だけを変えて「速そうに見せる」簡易的な演出。

c# DashState.cs

```
1 public class DashState : IState
2 {
3     private MarioCharacter mario;
4     public DashState(MarioCharacter mario)
5     {
6         this.mario = mario;
7     }
8     public void Enter()
9     {
10         mario.anim.Play("run-Animation");
11         mario.anim.speed = 1.5f;
12     }
13     public void Exit()
14     {
15         mario.anim.speed = 1f;
16     }
17 }
```

DashStateの実装②：加速するUpdate

- `Mathf.MoveTowards`で、今の速さから`dashSpeed`へ、`dashAcceleration`の分だけ毎フレーム近づけていく。

c# DashState.cs (Update)

```
1 private float currentSpeed;
2
3 public void Update()
4 {
5     float input = Input.GetAxisRaw("Horizontal");
6     currentSpeed = Mathf.MoveTowards(
7         currentSpeed, mario.dashSpeed, mario.dashAcceleration * Time.deltaTime);
8     mario.rb.linearVelocity = new Vector2(
9         input * currentSpeed, mario.rb.linearVelocity.y);
10
11     if (Mathf.Abs(input) < 0.1f)
12     {
13         mario.stateMachine.TransitionTo(mario.walkState);
14     }
15 }
```

ダッシュへの遷移トリガー

- WalkStateのUpdateに、ダッシュボタンが押されているかどうかの分岐を1つ追加するだけでよい。

c# MarioCharacter.cs (追記)

```
1 public float dashSpeed = 9f;  
2 public float dashAcceleration = 20f;
```

c# WalkState.cs (追記案)

```
1 // WalkState.Updateのステート切り替えをしているところに追加  
2 else if (Input.GetKey(KeyCode.LeftShift))  
3 {  
4     mario.stateMachine.TransitionTo(mario.dashState);  
5 }
```

問題

ジャンプダッシュの実装

- 現状ダッシュ中にジャンプができない状態になっているのでダッシュ中も同様にジャンプできるようにする

注意

Exitで速度を戻し忘れると、Dash以外の状態でもアニメーションが速いままになってしまいます。

STEP 2

ヒップドロップにディレイを追加

始まりと終わりに、間（ま）を作る

ヒップドロップのディレイとは

- 今のヒップドロップは、入力した瞬間に落下が始まり、着地した瞬間にIdleへ戻る
- 本家マリオ的には、「踏み込む溜め」と「着地後の硬直」があることで、動きに重みが出る
- 開始ディレイ：入力してから実際に落下し始めるまでの短い時間
- 終了ディレイ：着地してから、次の操作を受け付けるまでの短い時間

HipDropStateの改良設計

- 内部にタイマーとフラグ（着地済みかどうか）を持たせ、フェーズを区切って管理する
- ① 開始ディレイ中（タメ） → ② 落下中 → ③ 着地後の終了ディレイ中（硬直） → ④ Idleへ
- Updateの中で「今のフェーズか」を、タイマーの値と着地フラグで判断する

POINT

1つのStateの中に小さな「フェーズ」を持たせる書き方。State自体を増やすやり方との違いも授業で触れます。

HipDropStateの実装①：開始ディレイ

c# MarioCharacter.cs (追記)

```
1 public float hipDropStartDelay = 0.15f;  
2 public float hipDropEndDelay = 0.2f;
```

c# HipDropState.cs

```
1 private float timer;  
2 private bool hasLanded;  
3  
4 public void Enter()  
5 {  
6     timer = mario.hipDropStartDelay;  
7     hasLanded = false;  
8 }
```

HipDropStateの実装②：Update前半

- タイマーが残っている間はタメの最中。0になった瞬間に、実際の落下速度を与える。

c# HipDropState.cs (Update前半)

```
1 public void Update()
2 {
3     if (timer > 0)
4     {
5         timer -= Time.deltaTime;
6         if (timer <= 0)
7         {
8             mario.rb.linearVelocity = new Vector2(0f, -mario.hipDropSpeed);
9         }
10 }
```

HipDropStateの実装③：Update後半

- 着地した瞬間に終了ディレイを開始し、そのディレイが終わったらIdleへ戻る。

```
 HipDropState.cs  
(Update後半)  
  
11     if (!hasLanded && mario.isGrounded)  
12     {  
13         hasLanded = true;  
14         timer = mario.hipDropEndDelay;  
15     }  
16     else if (hasLanded)  
17     {  
18         timer -= Time.deltaTime;  
19         if (timer <= 0)  
20         {  
21             mario.stateMachine.TransitionTo(mario.idleState);  
22         }  
22     }  
23 }
```

STEP 3

壁キック & 壁張り付き

壁にくっついて、ゆっくり降りる・蹴って跳ぶ



壁アクションとは

- 壁張り付き (Wall Cling) : 壁に触れながら落下しているとき、通常より少しずつ・ゆっくり降下する
- 壁キック (Wall Kick) : 壁張り付き中にジャンプ入力をする、壁と逆方向へ跳ね返る
- 新しい状態 WallSlideState を1つ追加し、Fall/Jumpから壁接触時に遷移させる

WallSlideStateの実装①：構築とEnter

c# MarioCharacter.cs (追記)

```
1 public bool isTouchingWall;  
2 public float wallSlideSpeed = 2f;  
3 public float wallJumpForce = 6f;  
4 [SerializeField] public float rayRangeHolizon = 0.2f;
```

c# WallSlideState.cs

```
1 public class WallSlideState : IState  
2 {  
3     private MarioCharacter mario;  
4     public WallSlideState(MarioCharacter mario)  
5     {  
6         this.mario = mario;  
7     }  
8     public void Enter()  
9     {  
10         mario.anim.Play("fall-Animation");  
11         mario.sr.flipX = (Input.GetAxisRaw("Horizontal") > 0) ? true : false;  
12     }
```

WallSlideStateの実装②：ゆっくり降下

- 落下速度に上限 (wallSlideSpeed) をかけて、通常のFallよりゆっくり降りるようにする。

c# WallSlideState.cs (Update前半)

```
1 public void Update()  
2 {  
3     float y = Mathf.Max(  
4         mario.rb.linearVelocity.y, -mario.wallSlideSpeed);  
5     mario.rb.linearVelocity = new Vector2(mario.rb.linearVelocity.x, y);
```

WallSlideStateの実装③：壁キックと離脱

- 縦方向の速度はここでは触らない。JumpStateのEnterがjumpPowerで上書きしてくれるので、横方向だけ与えればよい。

c# WallSlideState.cs (Update後半)

```
1     if (Input.GetButtonDown("Jump"))
2     {
3         float direction = -Input.GetAxisRaw("Horizontal");
4         mario.rb.linearVelocity = new Vector2(
5             direction * mario.wallJumpForce, mario.rb.linearVelocity.y);
6         mario.stateMachine.TransitionTo(mario.jumpState);
7     }
8     else if (!mario.isTouchingWall)
9     {
10        mario.stateMachine.TransitionTo(mario.fallState);
11    }
12    else if (mario.isGrounded)
13    {
14        mario.stateMachine.TransitionTo(mario.idleState);
15    }
16 }
17 public void Exit() { }
```

壁張り付きへの遷移トリガー

- 落下中（Jump・Fall）に壁へ触れたら、WallSlideStateへ遷移する。

c# JumpState.cs / FallState.cs (追記案)

```
1 // JumpState.cs / FallState.cs のUpdateに追加
2 if (mario.isTouchingWall && mario.rb.linearVelocity.y < 0)
3 {
4     mario.stateMachine.TransitionTo(mario.wallSlideState);
5 }
```

壁判定の作成

- MarioCharacterで毎フレーム壁判定をする。

c# MarioCharacter.cs (追記)

```
1
2 public void Update()
3 {
4     CheckGround();
5     CheckWall();
6     stateMachine.Update();
7 }
```

壁判定の作成

- MarioCharacterで毎フレーム壁判定をする。

c# MarioCharacter.cs (追記)

```
1 private void CheckWall()  
2 {  
3     int layerMask = ~(1 << LayerMask.NameToLayer("Player"));  
4     float dir = Input.GetAxisRaw("Horizontal");  
5     RaycastHit2D hit = Physics2D.Raycast(transform.position, Vector2.right * dir, rayRange, layerMask);  
6     isTouchingWall = hit.collider != null;  
7 }
```

このコマのまとめ

- ダッシュ：WalkStateと似ているが別の状態として、DashStateを新設した
- ヒップドロップの強化：1つのStateの中にタイマーとフラグでフェーズを持たせた
- 壁キック&壁張り付き：WallSlideStateを新設し、Jump/Fallから壁接触で遷移させた
- 状態が増えても、既存のIdle/Walk/Jump/Fallにはほとんど手を入れずに済むことを確認する

お疲れさまでした

ダッシュ・ヒップドロップ・壁キックが加わり、
さらに本家マリオらしい動きに近づきました。